Домашнее задание 4. Отказоустойчивость с Apache ZooKeeper

Я поднял ансамбль из трёх серверов ZooKeeper 3.9.6 в Docker и посмотрел, как он ведёт себя при потере одного и двух серверов. Потом написал координатор расчёта пар навыков из третьего задания и запустил две его копии с тайм-аутом сессии 10 секунд. Лидера я замораживал на 40 секунд посреди расчёта. Запасной координатор стал лидером, досчитал остаток с чекпоинтов, результат совпал с ДЗ 3 байт в байт. Это я сделал два раза: без защиты по эпохе и с ней. Без защиты проснувшийся лидер записал результат второй раз, с защитой его запись отклонена

У длинных команд показаны только нужные строки. Скриншоты сделаны с веб-консоли RustFS

1. Стенд и устройство координатора

Компонент	**Версия и параметры**
ОС и среда	Windows 11, PowerShell 7, Docker 29.1
ZooKeeper	3.9.6, три контейнера, Java 17 (Temurin 17.0.20.1) внутри образа
Хранилище	RustFS 1.0.0, бакет hh-lake с данными и результатом из ДЗ 3
Координатор	Python 3.13, kazoo 2.11.0, boto3 1.43.123

Расчёт тот же, что в ДЗ 3 для стека 2, это пары навыков вакансий варианта 9. Файл в хранилище делится на 51 чанк по 16 МБ, чанки считают четыре процесса-маппера. После каждого чанка маппер пишет чекпоинт в бакет, это маленький JSON с частичными счётчиками пар

Выбор лидера сделан через ZooKeeper. Каждая копия создаёт эфемерный последовательный узел в очереди, лидером становится копия с наименьшим номером, остальные ждут исчезновения узла перед собой. Новый лидер увеличивает счётчик эпохи в отдельном узле и запоминает версию узла-указателя. Указатель хранит ссылку на опубликованный результат. Когда чанки кончились, лидер складывает все чекпоинты, пишет результат в бакет в папку своей эпохи и обновляет указатель

Проверку "я всё ещё лидер" в цикле расчёта я не делал. Процесс, который заморозили, о потере лидерства не знает, и нужная ситуация получается сама. После возобновления лидер пропускает чекпоинты, которые уже есть, досчитывает то, что было у его мапперов в работе, и публикует результат

Режим записи указателя задаёт переменная FENCING. При FENCING=0 запись идёт без проверки версии. При FENCING=1 лидер пишет только если версия указателя такая же, как была при его выборе

2. Этапы 1 и 2: ансамбль

Серверы подняты из образа с Java 17. У каждого свой конфиг и своя папка данных. Строки `server.N` одинаковые, а порты клиентов (2181, 2182, 2183) и порты обмена между серверами разные. В папке данных лежит файл myid с номером сервера. Каждый контейнер запускает сервер штатной командой zkServer.sh start-foreground, она видна в настройках контейнера:

```
PS C:\Users\memel\Desktop\Материалы, лабы\5 семестр\Инженерия Данных\data-
engineering-lab4> docker compose ps --format "table
{{.Name}}\t{{.Image}}\t{{.Ports}}"
NAME      IMAGE             PORTS
zk1-hw4   zookeeper:3.9.6   0.0.0.0:2181->2181/tcp, [::]:2181->2181/tcp
zk2-hw4   zookeeper:3.9.6   0.0.0.0:2182->2182/tcp, [::]:2182->2182/tcp
zk3-hw4   zookeeper:3.9.6   0.0.0.0:2183->2183/tcp, [::]:2183->2183/tcp

PS C:\Users\memel\Desktop\Материалы, лабы\5 семестр\Инженерия Данных\data-
engineering-lab4> docker inspect zk1-hw4 --format "{{.Config.Entrypoint}}
{{.Config.Cmd}}"
[/docker-entrypoint.sh] [zkServer.sh start-foreground]

PS C:\Users\memel\Desktop\Материалы, лабы\5 семестр\Инженерия Данных\data-
engineering-lab4> docker exec zk1-hw4 sh -c "java -version 2>&1 | head -2 | tail
-1"
OpenJDK Runtime Environment Temurin-17.0.20.1+1 (build 17.0.20.1+1)

PS C:\Users\memel\Desktop\Материалы, лабы\5 семестр\Инженерия Данных\data-
engineering-lab4> Get-Content zk\conf\zk1\zoo.cfg
tickTime=2000
initLimit=10
syncLimit=5
dataDir=/data/zk1
clientPort=2181
admin.enableServer=false
4lw.commands.whitelist=srvr,stat,mntr,ruok
server.1=zk1:2888:3888
server.2=zk2:2889:3889
server.3=zk3:2890:3890

PS C:\Users\memel\Desktop\Материалы, лабы\5 семестр\Инженерия Данных\data-
engineering-lab4> Get-Content zk\conf\zk2\zoo.cfg | Select-String
"dataDir|clientPort"

dataDir=/data/zk2
clientPort=2182

PS C:\Users\memel\Desktop\Материалы, лабы\5 семестр\Инженерия Данных\data-
engineering-lab4> Get-Content zk\conf\zk3\zoo.cfg | Select-String
"dataDir|clientPort"

dataDir=/data/zk3
clientPort=2183

PS C:\Users\memel\Desktop\Материалы, лабы\5 семестр\Инженерия Данных\data-
engineering-lab4> 1..3 | ForEach-Object { "zk$_ myid = " + (Get-Content
zk\data\zk$_\myid) }
zk1 myid = 1
zk2 myid = 2
zk3 myid = 3

PS C:\Users\memel\Desktop\Материалы, лабы\5 семестр\Инженерия Данных\data-
```

```
engineering-lab4> curl.exe -s -o NUL -w "health %{http_code}`n"
http://localhost:9200/health
health 200

PS C:\Users\memel\Desktop\Материалы, лабы\5 семестр\Инженерия Данных\data-
engineering-lab4> rclone copy ..\task2\data\it_vacancies_2024.csv rustfs:hh-
lake/raw/ --stats-one-line --stats 30s -v --log-format nolevel 2>&1
S3 bucket hh-lake path raw: Bucket "hh-lake" created with ACL ""
      340 MiB / 808.353 MiB, 42%, 11.451 MiB/s, ETA 40s
      635 MiB / 808.353 MiB, 79%, 9.826 MiB/s, ETA 17s
it_vacancies_2024.csv: Multi-thread Copied (new)
  808.353 MiB / 808.353 MiB, 100%, 5.477 MiB/s, ETA 0s

PS C:\Users\memel\Desktop\Материалы, лабы\5 семестр\Инженерия Данных\data-
engineering-lab4> rclone lsf rustfs:hh-lake --recursive --files-only --format
"ps"
raw/it_vacancies_2024.csv;847619384

PS C:\Users\memel\Desktop\Материалы, лабы\5 семестр\Инженерия Данных\data-
engineering-lab4> rclone md5sum rustfs:hh-lake/raw/it_vacancies_2024.csv
45041a115287e29b41fc8f82818ec5a6  it_vacancies_2024.csv

PS C:\Users\memel\Desktop\Материалы, лабы\5 семестр\Инженерия Данных\data-
engineering-lab4> (Get-FileHash ..\task2\data\it_vacancies_2024.csv -Algorithm
MD5).Hash.ToLower()
45041a115287e29b41fc8f82818ec5a6
```

Дальше я посмотрел роли, остановил одного участника (на тот момент лидера), потом второго. С двумя остановленными серверами оставшийся сам себя называет неработающим, а клиент получает ConnectionLoss. После запуска обоих серверов ансамбль снова отвечает:

```
PS C:\Users\memel\Desktop\Материалы, лабы\5 семестр\Инженерия Данных\data-
engineering-lab4> docker exec zk1-hw4 zkServer.sh status
Client port found: 2181. Client address: localhost. Client SSL: false.
Mode: follower

PS C:\Users\memel\Desktop\Материалы, лабы\5 семестр\Инженерия Данных\data-
engineering-lab4> 1..3 | ForEach-Object { "zk$_ - " + (docker exec zk$_-hw4
zkServer.sh status 2>&1 | Select-Object -Last 1) }
zk1 - Mode: follower
zk2 - Mode: follower
zk3 - Mode: leader

PS C:\Users\memel\Desktop\Материалы, лабы\5 семестр\Инженерия Данных\data-
engineering-lab4> docker stop zk3-hw4
zk3-hw4

PS C:\Users\memel\Desktop\Материалы, лабы\5 семестр\Инженерия Данных\data-
engineering-lab4> 1..3 | ForEach-Object { "zk$_ - " + (docker exec zk$_-hw4
zkServer.sh status 2>&1 | Select-Object -Last 1) }
zk1 - Mode: follower
zk2 - Mode: leader
```

```
zk3 - Error response from daemon: container
05928cdf552393222b61a7a8b602751fb203f89f8889037af6d01ebb2fc351de is not running

PS C:\Users\memel\Desktop\Материалы, лабы\5 семестр\Инженерия Данных\data-
engineering-lab4> uv run python tools\zkcmd.py zk1 "ls /"
[zookeeper]

PS C:\Users\memel\Desktop\Материалы, лабы\5 семестр\Инженерия Данных\data-
engineering-lab4> docker stop zk2-hw4
zk2-hw4

PS C:\Users\memel\Desktop\Материалы, лабы\5 семестр\Инженерия Данных\data-
engineering-lab4> 1..3 | ForEach-Object { "zk$_ - " + (docker exec zk$_-hw4
zkServer.sh status 2>&1 | Select-Object -Last 1) }
zk1 - Error contacting service. It is probably not running.
zk2 - Error response from daemon: container
3261fa9d5e0047e282183dc75924a8d965353b23622c839fdf10724246800b62 is not running
zk3 - Error response from daemon: container
05928cdf552393222b61a7a8b602751fb203f89f8889037af6d01ebb2fc351de is not running

PS C:\Users\memel\Desktop\Материалы, лабы\5 семестр\Инженерия Данных\data-
engineering-lab4> docker exec zk1-hw4 sh -c "echo srvr | nc localhost 2181"
This ZooKeeper instance is not currently serving requests

PS C:\Users\memel\Desktop\Материалы, лабы\5 семестр\Инженерия Данных\data-
engineering-lab4> uv run python tools\zkcmd.py zk1 "ls /" 15
KeeperErrorCode = ConnectionLoss for /

PS C:\Users\memel\Desktop\Материалы, лабы\5 семестр\Инженерия Данных\data-
engineering-lab4> docker start zk2-hw4 zk3-hw4
zk2-hw4
zk3-hw4

PS C:\Users\memel\Desktop\Материалы, лабы\5 семестр\Инженерия Данных\data-
engineering-lab4> 1..3 | ForEach-Object { "zk$_ - " + (docker exec zk$_-hw4
zkServer.sh status 2>&1 | Select-Object -Last 1) }
zk1 - Mode: follower
zk2 - Mode: leader
zk3 - Mode: follower

PS C:\Users\memel\Desktop\Материалы, лабы\5 семестр\Инженерия Данных\data-
engineering-lab4> uv run python tools\zkcmd.py zk1 "ls /"
[zookeeper]
```

Пока работают два сервера из трёх, ансамбль выбрал нового лидера и отвечает на запросы. Когда остался один, кворума нет: сервер отвечает, что не обслуживает запросы, и ls / даёт ConnectionLoss

Для эфемерного узла во втором окне zkCli я создал узел /demo-ephemeral с флагом -e, в первом окне он виден на другом сервере. После закрытия второго окна сессия кончилась и узел исчез, get отвечает Node does not exist:

```
PS C:\Users\memel\Desktop\Материалы, лабы\5 семестр\Инженерия Данных\data-
engineering-lab4> uv run python tools\ephemeral_demo.py

PS C:\Users\memel\Desktop\Материалы, лабы\5 семестр\Инженерия Данных\data-
engineering-lab4> docker exec -i zk2-hw4 zkCli.sh -server zk2:2182     # окно 2
[zk: zk2:2182(CONNECTED) 0] create -e /demo-ephemeral mematvey
Created /demo-ephemeral
[zk: zk2:2182(CONNECTED) 1] stat /demo-ephemeral
cZxid = 0x300000003
mZxid = 0x300000003
pZxid = 0x300000003
cversion = 0
dataVersion = 0
aclVersion = 0
ephemeralOwner = 0x2000047fd540000
dataLength = 8
numChildren = 0

PS C:\Users\memel\Desktop\Материалы, лабы\5 семестр\Инженерия Данных\data-
engineering-lab4> docker exec zk1-hw4 zkCli.sh -server zk1:2181 ls /     # окно 1
[demo-ephemeral, zookeeper]

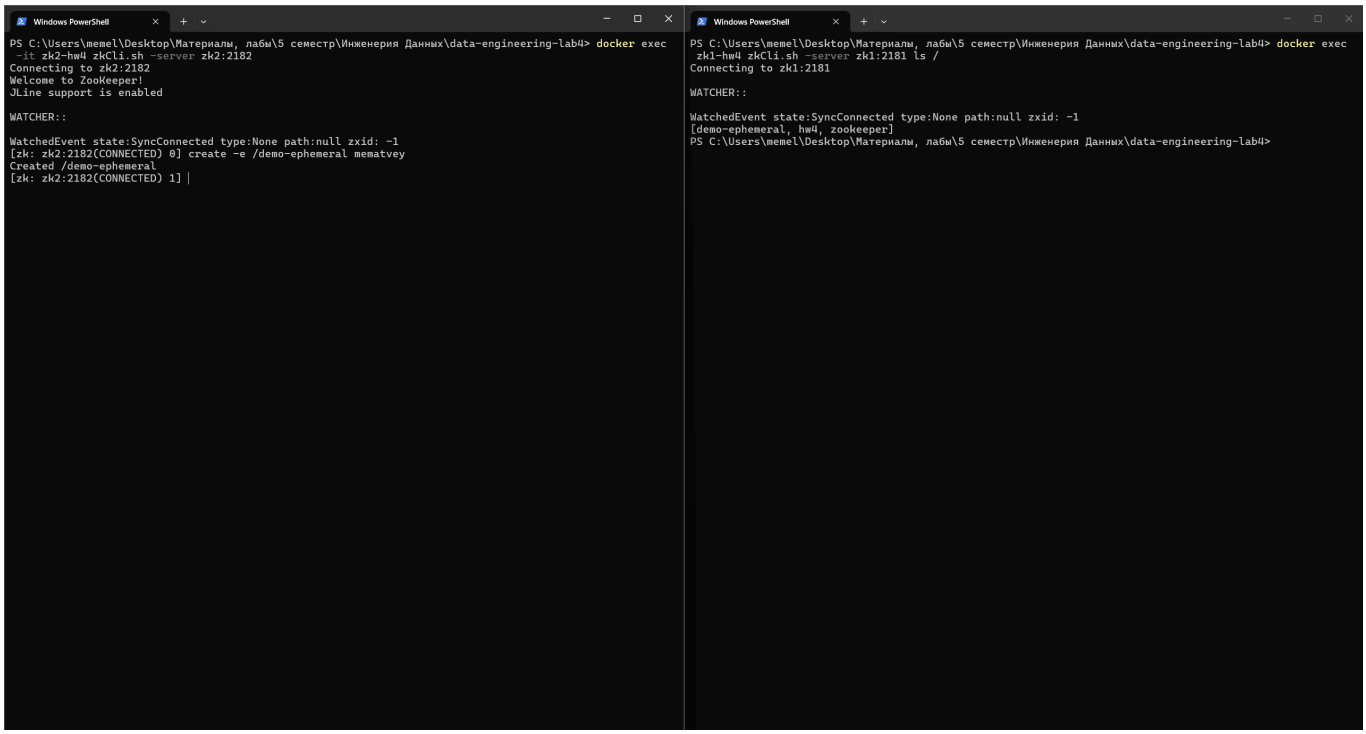
PS C:\Users\memel\Desktop\Материалы, лабы\5 семестр\Инженерия Данных\data-
engineering-lab4> docker exec zk1-hw4 zkCli.sh -server zk1:2181 get /demo-
ephemeral     # окно 1
mematvey

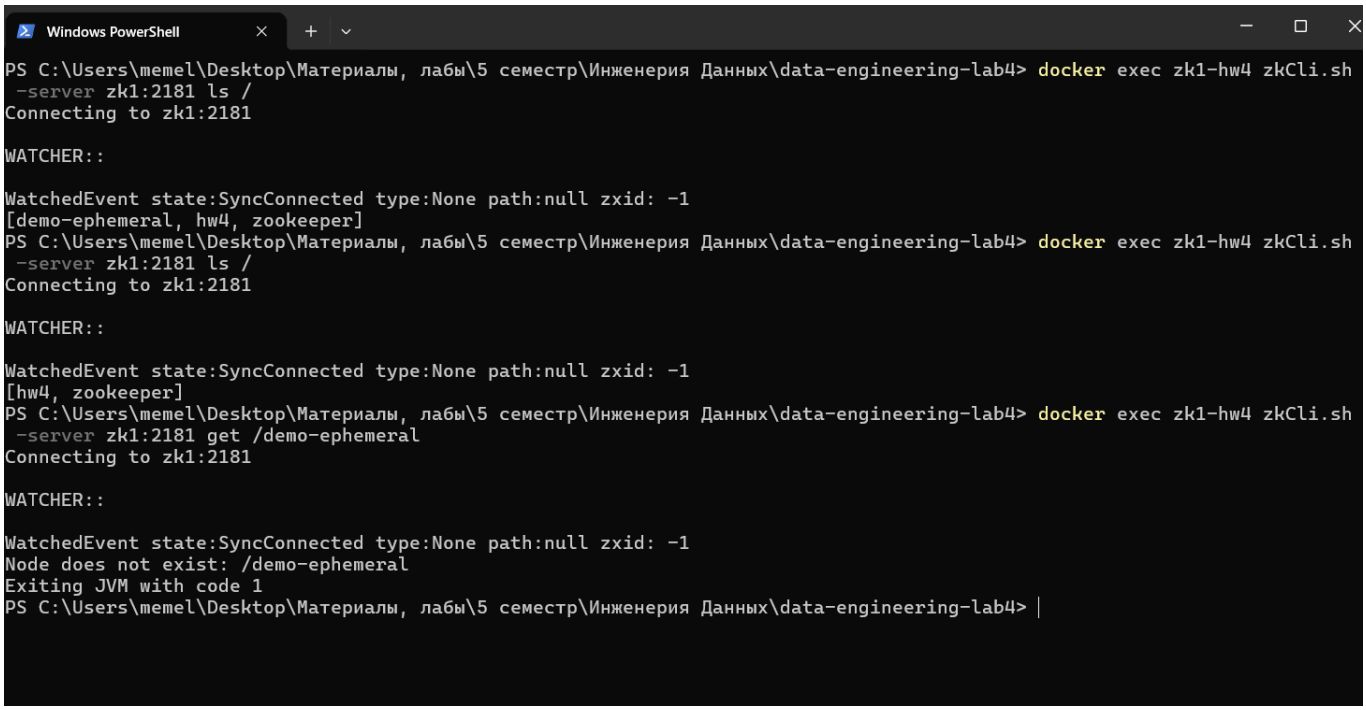
[zk: zk2:2182(CONNECTED) 2] quit     # закрываю окно 2
WatchedEvent state:Closed type:None path:null zxid: -1

PS C:\Users\memel\Desktop\Материалы, лабы\5 семестр\Инженерия Данных\data-
engineering-lab4> docker exec zk1-hw4 zkCli.sh -server zk1:2181 ls /     # окно 1
[zookeeper]

PS C:\Users\memel\Desktop\Материалы, лабы\5 семестр\Инженерия Данных\data-
engineering-lab4> docker exec zk1-hw4 zkCli.sh -server zk1:2181 get /demo-
ephemeral     # окно 1
Node does not exist: /demo-ephemeral
```

Те же шаги я повторил руками в двух окнах PowerShell. Пока второе окно открыто, узел виден из первого. После выхода из второго окна список его уже не содержит, и get отвечает Node does not exist





3. Этап 3: приостановка лидера без защиты

Сценарий запускает копию A, ждёт, пока она станет лидером, и запускает копию B. Когда у A готово 20 чекпоинтов из 51, сценарий замораживает все процессы A. Через 40 секунд он размораживает их. Время в выводе это секунды от старта сценария:

```
PS C:\Users\memel\Desktop\Материалы, лабы\5 семестр\Инженерия Данных\data-
engineering-lab4> uv run python src\run_scenario.py --run nofence --fencing 0
[t=   0.00 с] [сценарий] запуск FENCING=0
[t=   1.87 с] [сценарий] A (pid 30372) лидер, B (pid 12724) запасной
[t=  17.47 с] [сценарий] приостановил лидера A (процессов 6), у него чекпоинтов:
20
[t=  27.40 с] [сценарий] запасной B стал лидером в t=27.29 с, смена лидера заняла
```

```
9.8 с после паузы
[t=  57.47 с] [сценарий] возобновил лидера A после паузы в 40.0 с
[t=  59.78 с] [сценарий] оба координатора завершились: A код 0, B код 0
[t=  59.82 с] [сценарий] указатель: версия 2, данные {"epoch": 1, "leader": "A",
"key": "results/zk/nofence/epoch_1/stack_2.csv", "md5":
"f47fc1deadb3f8173e6ab9833c9a7d30"}
[t=  60.09 с] [сценарий] результатов в бакете: 2:
results/zk/nofence/epoch_1/stack_2.csv, results/zk/nofence/epoch_2/stack_2.csv
```

Ключевые строки из журналов обеих копий, отсортированные по времени:

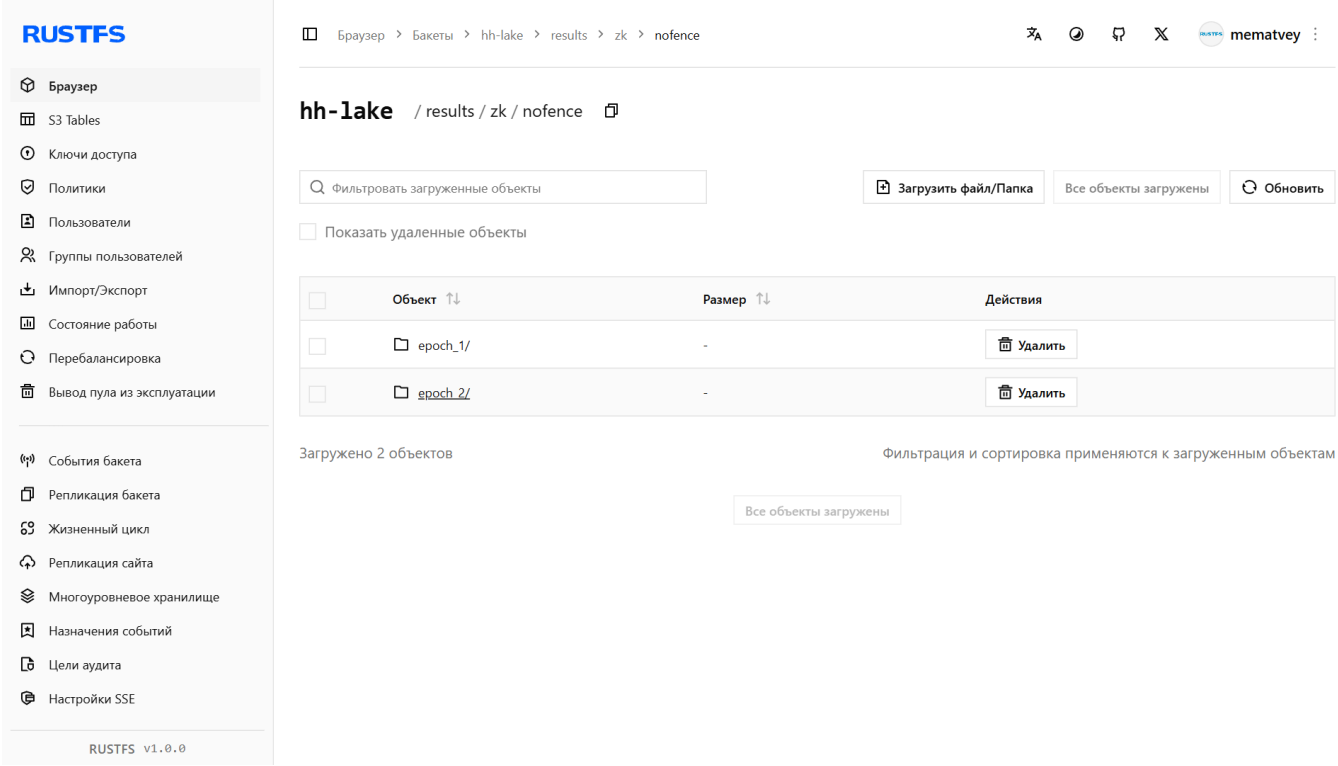
```
PS C:\Users\memel\Desktop\Материалы, лабы\5 семестр\Инженерия Данных\data-
engineering-lab4> Get-Content logs\nofence_A.log, logs\nofence_B.log -Encoding
utf8 | Where-Object { $_ -match 'стал лидером|нашёл чекпоинты|записал
результат|BadVersionError|состояние сессии|пересчитывать' } | Sort-Object {
[double][regex]::Match($_, 't=\s*([\d.]+)').Groups[1].Value }
[t=   0.74 с] [A] состояние сессии: CONNECTED
[t=   0.87 с] [A] стал лидером: эпоха 1, версия указателя при выборе 0
[t=   1.82 с] [B] состояние сессии: CONNECTED
[t=  27.29 с] [B] стал лидером: эпоха 2, версия указателя при выборе 0
[t=  29.22 с] [B] нашёл чекпоинты: 23 из 51, продолжаю с остатка
[t=  44.75 с] [B] эпоха 2: записал результат
results/zk/nofence/epoch_2/stack_2.csv, указатель теперь версии 1
[t=  44.77 с] [B] состояние сессии: LOST
[t=  57.47 с] [A] состояние сессии: SUSPENDED
[t=  57.49 с] [A] состояние сессии: LOST
[t=  57.51 с] [A] состояние сессии: CONNECTED
[t=  58.11 с] [A] эпоха 1: чекпоинты 26 чанков уже лежат в бакете, пересчитывать
не стал
[t=  59.62 с] [A] эпоха 1: записал результат
results/zk/nofence/epoch_1/stack_2.csv, указатель теперь версии 2
[t=  59.63 с] [A] состояние сессии: LOST
```

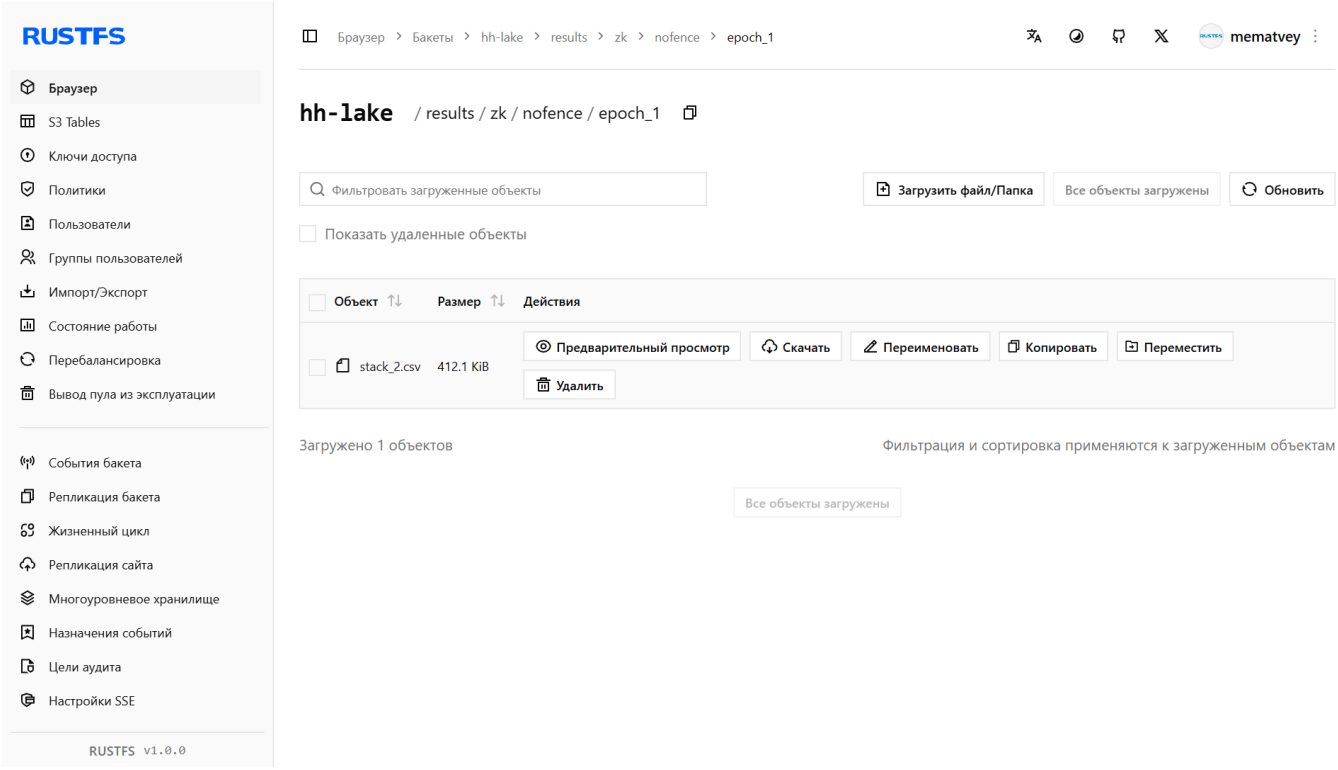
Лидер сменился через 9,8 с после приостановки. Это близко к тайм-ауту сессии в 10 секунд. Сервер ждёт столько без сигналов от клиента и только потом удаляет его эфемерный узел. B увидел, что узла перед ним больше нет, получил эпоху 2 и нашёл 23 готовых чекпоинта. Три лишних чекпоинта появились, похоже, пока замораживались процессы A, и мапперы успели их дописать. Остальные 28 чанков он досчитал сам и записал результат раньше, чем A проснулся. Это первая строка "записал результат", указатель стал версии 1

После возобновления A получил события SUSPENDED, LOST и снова CONNECTED. Сессия у него была потеряна, но работу он продолжил. Он пропустил 26 чанков, потому что их чекпоинты уже лежали в бакете. Ещё 2 чанка были у его мапперов в работе на момент паузы, их он посчитал заново, хотя B уже записал чекпоинты для них. Лишней работой это было, но результат не изменился. Потом A записал результат второй раз и поднял указатель до версии 2. Теперь указатель ведёт на результат старого лидера из эпохи 1, а в бакете лежат оба результата

Результаты сверены с MapReduce из ДЗ 3, оба набора читаются из бакета. Содержимое обоих файлов совпало с ДЗ 3 побайтово:

```
PS C:\Users\memel\Desktop\Материалы, лабы\5 семестр\Инженерия Данных\data-
engineering-lab4> uv run python src\verify.py --run nofence
ДЗ 3: results/mapreduce/stack_2.csv: 14646 пар, сумма 57782, md5
f47fc1deadb3f8173e6ab9833c9a7d30
results/zk/nofence/epoch_1/stack_2.csv: 14646 пар, сумма 57782, md5
f47fc1deadb3f8173e6ab9833c9a7d30
  только в ДЗ 3: 0, только здесь: 0, разные счётчики: 0, файлы побайтово равны:
True
results/zk/nofence/epoch_2/stack_2.csv: 14646 пар, сумма 57782, md5
f47fc1deadb3f8173e6ab9833c9a7d30
  только в ДЗ 3: 0, только здесь: 0, разные счётчики: 0, файлы побайтово равны:
True
ИТОГ: СОВПАЛО
```





4. Этап 4: приостановка лидера с защитой по эпохе

Сценарий тот же, но с FENCING=1. Лидер при выборе запоминает версию указателя, и записать указатель может, только если версия не изменилась. В этом запуске у обоих лидеров при выборе версия была 0:

```
PS C:\Users\memel\Desktop\Материалы, лабы\5 семестр\Инженерия Данных\data-
engineering-lab4> uv run python src\run_scenario.py --run fence --fencing 1
[t=   0.00 c] [сценарий] запуск FENCING=1
[t=   1.78 c] [сценарий] A (pid 28576) лидер, B (pid 6348) запасной
[t=  12.16 c] [сценарий] приостановил лидера A (процессов 6), у него чекпоинтов:
20
[t=  21.47 c] [сценарий] запасной B стал лидером в t=21.41 c, смена лидера заняла
9.2 c после паузы
[t=  52.17 c] [сценарий] возобновил лидера A после паузы в 40.0 c
[t=  55.84 c] [сценарий] оба координатора завершились: A код 3, B код 0
[t=  55.89 c] [сценарий] указатель: версия 1, данные {"epoch": 2, "leader": "B",
"key": "results/zk/fence/epoch_2/stack_2.csv", "md5":
"f47fc1deadb3f8173e6ab9833c9a7d30"}
[t=  56.01 c] [сценарий] результатов в бакете: 1:
results/zk/fence/epoch_2/stack_2.csv
```

Строки из журналов обеих копий:

```
PS C:\Users\memel\Desktop\Материалы, лабы\5 семестр\Инженерия Данных\data-
engineering-lab4> Get-Content logs\fence_A.log, logs\fence_B.log -Encoding utf8 |
Where-Object { $_ -match 'стал лидером|нашёл чекпоинты|записал
результат|BadVersionError|состояние сессии|пересчитывать' } | Sort-Object {
[double][regex]::Match($_, 't=\s*([\d.]+)').Groups[1].Value }
[t=   0.78 c] [A] состояние сессии: CONNECTED
```

```
[t=   0.87 с] [A] стал лидером: эпоха 1, версия указателя при выборе 0
[t=   1.73 с] [B] состояние сессии: CONNECTED
[t=  21.41 с] [B] стал лидером: эпоха 2, версия указателя при выборе 0
[t=  23.10 с] [B] нашёл чекпоинты: 20 из 51, продолжаю с остатка
[t=  41.07 с] [B] эпоха 2: записал результат
results/zk/fence/epoch_2/stack_2.csv, указатель теперь версии 1
[t=  41.08 с] [B] состояние сессии: LOST
[t=  52.17 с] [A] состояние сессии: SUSPENDED
[t=  52.18 с] [A] состояние сессии: LOST
[t=  52.20 с] [A] состояние сессии: CONNECTED
[t=  53.17 с] [A] эпоха 1: чекпоинты 27 чанков уже лежат в бакете, пересчитывать
не стал
[t=  55.68 с] [A] эпоха 1: запись указателя отклонена, BadVersionError: ждал
версию 0, в ZooKeeper версия 1
[t=  55.69 с] [A] состояние сессии: LOST
```

Смена лидера заняла 9,2 с. Запасной B записал результат, указатель стал версии 1. Старый лидер A после возобновления пришёл с версией 0, и ZooKeeper вернул BadVersionError. Запись отклонена, A удалил свой результат и завершился с кодом 3. В бакете остался один результат от эпохи 2, указатель остался версии 1 от B. Этот результат тоже совпал с ДЗ 3:

```
PS C:\Users\memel\Desktop\Материалы, лабы\5 семестр\Инженерия Данных\data-
engineering-lab4> uv run python src\verify.py --run fence
ДЗ 3: results/mapreduce/stack_2.csv: 14646 пар, сумма 57782, md5
f47fc1deadb3f8173e6ab9833c9a7d30
results/zk/fence/epoch_2/stack_2.csv: 14646 пар, сумма 57782, md5
f47fc1deadb3f8173e6ab9833c9a7d30
  только в ДЗ 3: 0, только здесь: 0, разные счётчики: 0, файлы побайтово равны:
True
ИТОГ: СОВПАЛО
```

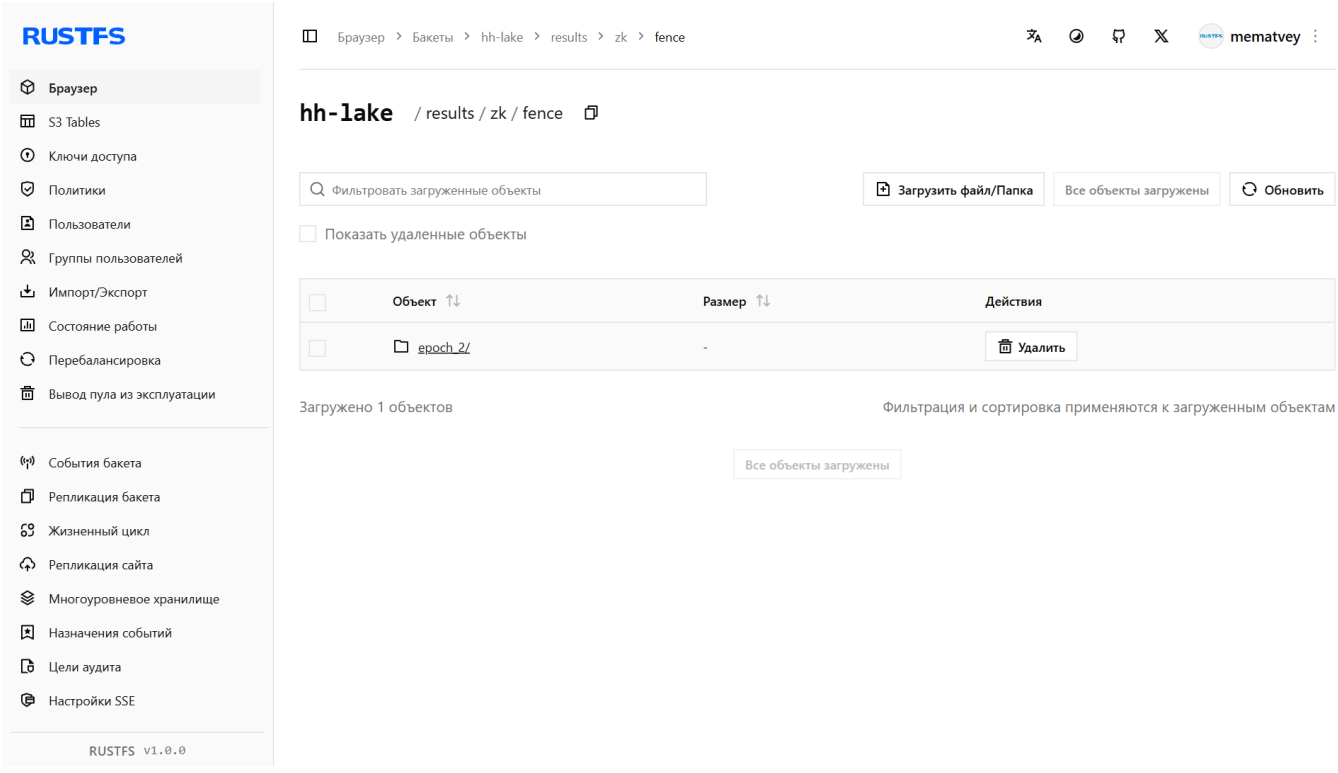


Таблица 1. Сравнение двух запусков

Показатель	**Без защиты**	**С защитой**
Чекпоинтов у лидера в момент паузы	20 из 51	20 из 51
Чекпоинтов нашёл запасной	23 из 51	20 из 51
Смена лидера, с	9,8	9,2
Строк "записал результат"	2	1
Строк BadVersionError	0	1
Версия указателя в конце	2	1
Указатель ведёт на результат	эпохи 1, лидер A	эпохи 2, лидер B
Результатов в бакете	2	1
Пар навыков, сумма вхождений	14 646, 57 782	14 646, 57 782
Совпало с ДЗ 3 побайтово	да	да

Время смены лидера я мерил по одному разу в каждом запуске. Разницу в 0,6 секунды я считаю не очень значимой, тк она похожа на случайный разброс момента, когда сервер заметил пропажу сессии

В этом сценарии результаты обоих лидеров одинаковые, и повторная запись ничего не портит. Если бы результат старого лидера отличался, например считался по старому снимку данных, без защиты указатель в конце вёл бы именно на него

5. Ответы на вопросы

Почему серверов три, а не два и не четыре

ZooKeeper отвечает, пока живо больше половины серверов. Для трёх это два сервера. Я остановил два из трёх, и оставшийся ответил, что не обслуживает запросы. С одним остановленным ансамбль работал

Два сервера хуже трёх - большинство там тоже два, и остановка любого из них выключает ансамбль. Отказов он держит столько же, сколько один сервер, а сломаться может по любому из двух. Четыре сервера дают большинство из трёх и снова допускают потерю только одного. Четвёртый сервер живучесть не повышает, но запись замедляет, ведь подтверждений нужно больше. При разрыве сети пополам по два сервера большинства не будет ни у одной половины. Нечётное число выгоднее: три переживают один отказ, пять два

Почему защиту по эпохе проверяет получатель записи, а не ZooKeeper

В моих запусках лидер A после возобновления узнал о потере сессии, в журнале есть LOST. Работу он всё равно продолжил и опубликовал результат, потому что в цикле расчёта нет проверки лидерства. Такая проверка тоже не спасла бы: между проверкой и самой записью лидерство может кончиться, а замороженный процесс не знает, сколько прошло времени. ZooKeeper не может отозвать запись, которую процесс отправляет в бакет. Он хранит узлы и сообщает, кто лидер. Решить, что запись устарела, может только тот, кто её принимает, тк он сравнивает эпоху или версию из запроса с последней известной

В моём стенде получатель указателя это сам ZooKeeper, он и выдал BadVersionError. Проверка работает потому, что узел сравнивает версию из запроса со своей. В запуске с защитой А успел записать свой файл в бакет до отказа, и мне пришлось удалять его после BadVersionError. Бакет такой проверки не делает. Если бы указатель принимало другое хранилище, ему пришлось бы самому хранить последнюю эпоху и отвергать записи со старой

Итоги

- Ансамбль из трёх серверов ZooKeeper собран, роли видны. Без одного сервера он работает, без двух нет, после запуска обоих серверов снова отвечает
- Эфемерный узел из второго окна zkCli исчез после закрытия окна
- Без защиты запасной координатор стал лидером через 9,8 с, досчитал остаток с 23 чекпоинтов, результат совпал с Д3 3. Проснувшийся лидер записал результат второй раз, указатель дошёл до версии 2
- С защитой смена лидера заняла 9,2 с, запись проснувшегося лидера отклонена с BadVersionError, результат один, указатель от запасного координатора
- Время смены лидера около 10 секунд, как и тайм-аут сессии
- Три сервера нужны ради кворума: два не переживают ни одного отказа, четыре дают то же, что три
- Защиту по эпохе проверяет получатель записи, лидер после паузы продолжает работу даже после потери сессии, а ZooKeeper не может остановить его запрос